

人工智能实验二实验报告：CNN 算法实现与应用

2023302855-梁景毓

实验要求

- 用MNIST数据集训练编写的CNN网络,要求记录每次迭代的损失值。
- 改变卷积神经网络卷积层和池化层数,观察分类准确率。思考网络层数的多少对分类准确率的影响。
- 改变卷积神经网络卷积核大小,观察分类准确率。思考网络卷积核大小对分类准确率的影响。

算法设计

卷积神经网络 (CNN) 通过“卷积”和“池化”两个主要操作来提取图像特征：

- 卷积层 (Convolutional Layer)**: 使用一组可学习的滤波器（卷积核）扫描输入图像，提取边缘、纹理等局部特征。
- 池化层 (Pooling Layer)**: 对特征图进行下采样（如最大池化 Max Pooling），减少数据维度，降低计算量，并增强模型对平移的鲁棒性。
- 全连接层 (Fully Connected Layer)**: 将提取到的高维特征展平，通过常规神经网络进行分类。

实验设计

1. CNN网络构建

我们首先实现一个基础的 CNN 网络，包含两层卷积，并记录训练过程中的 Loss 变化。

核心代码 (task1_basic_loss/main.py):

```
1  # Convolutional neural network (two convolutional layers)
2  class ConvNet(nn.Module):
3      def __init__(self):
4          super(ConvNet, self).__init__()
5          # Layer 1
6          self.layer1 = nn.Sequential(
7              nn.Conv2d(1, 16, kernel_size=5, stride=1, padding=2),
8              nn.BatchNorm2d(16),
9              nn.ReLU(),
10             nn.MaxPool2d(kernel_size=2, stride=2))
11         # Layer 2
12         self.layer2 = nn.Sequential(
13             nn.Conv2d(16, 32, kernel_size=5, stride=1, padding=2),
14             nn.BatchNorm2d(32),
15             nn.ReLU(),
16             nn.MaxPool2d(kernel_size=2, stride=2))
17         # Fully connected layer
18         self.fc = nn.Linear(7*7*32, 10)
19
20     def forward(self, x):
21         out = self.layer1(x)
22         out = self.layer2(out)
```

```
23         out = out.reshape(out.size(0), -1)
24         out = self.fc(out)
25         return out
```

运行结果 (Loss 记录):

```
1 Starting training...
2 Iteration, Loss
3 Epoch [1/5], Step [100/600], Loss: 0.2035
4 Epoch [1/5], Step [200/600], Loss: 0.1537
5 Epoch [1/5], Step [300/600], Loss: 0.0379
6 Epoch [1/5], Step [400/600], Loss: 0.0341
7 Epoch [1/5], Step [500/600], Loss: 0.0418
8 Epoch [1/5], Step [600/600], Loss: 0.1165
9 Epoch [2/5], Step [100/600], Loss: 0.0933
10 Epoch [2/5], Step [200/600], Loss: 0.0468
11 Epoch [2/5], Step [300/600], Loss: 0.0116
12 Epoch [2/5], Step [400/600], Loss: 0.0130
13 Epoch [2/5], Step [500/600], Loss: 0.0550
14 Epoch [2/5], Step [600/600], Loss: 0.0263
15 Epoch [3/5], Step [100/600], Loss: 0.0296
16 Epoch [3/5], Step [200/600], Loss: 0.0493
17 Epoch [3/5], Step [300/600], Loss: 0.0397
18 Epoch [3/5], Step [400/600], Loss: 0.0348
19 Epoch [3/5], Step [500/600], Loss: 0.0056
20 Epoch [3/5], Step [600/600], Loss: 0.0130
21 Epoch [4/5], Step [100/600], Loss: 0.0301
22 Epoch [4/5], Step [200/600], Loss: 0.0080
23 Epoch [4/5], Step [300/600], Loss: 0.0520
24 Epoch [4/5], Step [400/600], Loss: 0.0027
25 Epoch [4/5], Step [500/600], Loss: 0.0549
26 Epoch [4/5], Step [600/600], Loss: 0.0320
27 Epoch [5/5], Step [100/600], Loss: 0.0055
28 Epoch [5/5], Step [200/600], Loss: 0.0228
29 Epoch [5/5], Step [300/600], Loss: 0.0016
30 Epoch [5/5], Step [400/600], Loss: 0.0290
31 Epoch [5/5], Step [500/600], Loss: 0.0037
32 Epoch [5/5], Step [600/600], Loss: 0.0359
33 Training finished. Loss history saved to loss_history.txt
```

2. 改变卷积层数和池化层数

为了探究网络深度对性能的影响，我设计了 5 组实验：

- **1层 (Shallow):** 仅包含 1 个卷积层。
- **2层 (Standard):** 包含 2 个卷积层。
- **3层 (Deep):** 包含 3 个卷积层。
- **4层 (Deeper):** 包含 4 个卷积层。
- **5层 (Deepest):** 包含 5 个卷积层。

关键代码 (5层网络 `task2_depth_impact/cnn_depth_5.py`):

```

1  # 5-Layer CNN (VGG-style blocks)
2  class ConvNetDepth5(nn.Module):
3      def __init__(self):
4          super(ConvNetDepth5, self).__init__()
5
6          # Block 1: 28x28 -> 14x14 (2 convs)
7          self.block1 = nn.Sequential(
8              nn.Conv2d(1, 16, kernel_size=3, padding=1),
9              nn.BatchNorm2d(16),
10             nn.ReLU(),
11             nn.Conv2d(16, 32, kernel_size=3, padding=1),
12             nn.BatchNorm2d(32),
13             nn.ReLU(),
14             nn.MaxPool2d(2)
15         )
16
17         # Block 2: 14x14 -> 7x7 (2 convs)
18         self.block2 = nn.Sequential(
19             nn.Conv2d(32, 64, kernel_size=3, padding=1),
20             nn.BatchNorm2d(64),
21             nn.ReLU(),
22             nn.Conv2d(64, 64, kernel_size=3, padding=1),
23             nn.BatchNorm2d(64),
24             nn.ReLU(),
25             nn.MaxPool2d(2)
26         )
27
28         # Block 3: 7x7 -> 3x3 (1 conv + pool)
29         self.block3 = nn.Sequential(
30             nn.Conv2d(64, 128, kernel_size=3, padding=1),
31             nn.BatchNorm2d(128),
32             nn.ReLU(),
33             nn.MaxPool2d(2)
34         )
35
36         self.fc = nn.Linear(3*3*128, 10)
37
38     def forward(self, x):
39         out = self.block1(x)
40         out = self.block2(out)
41         out = self.block3(out)
42         out = out.reshape(out.size(0), -1)
43         out = self.fc(out)
44         return out

```

实验结果:

```

1  Running Task 2: Depth Impact (Shallow - 1 Layer)...
2  Accuracy of Shallow CNN (1 Layer) on 10000 test images: 98.33 %
3
4  Running Task 2: Depth Impact (Standard - 2 Layers)...
5  Accuracy of CNN (Depth 2) on 10000 test images: 98.97 %
6
7  Running Task 2: Depth Impact (Deep - 3 Layers)...
8  Accuracy of Deep CNN (3 Layers) on 10000 test images: 99.03 %

```

```
9
10 Running Task 2: Depth Impact (Deeper - 4 Layers)...
11 Accuracy of CNN (Depth 4) on 10000 test images: 98.97 %
12
13 Running Task 2: Depth Impact (Deepest - 5 Layers)...
14 Accuracy of CNN (Depth 5) on 10000 test images: 99.24 %
```

实验分析:

1. **浅层网络局限性:** 1层网络的准确率最低 (98.33%), 说明单层特征提取能力有限。
2. **深度提升性能:** 随着网络从1层增加到3层, 准确率有明显提升 (从 98.33% 到 99.03%) 。
3. **最优深度:** 在本实验中, 5层网络达到了最高的准确率 (99.24%)。这表明更深的网络结构能够提取更高级、更抽象的语义特征, 从而提升分类性能。虽然 4 层网络略有波动, 但整体趋势符合“深度网络通常优于浅层网络”的规律。

3. 改变卷积内核大小, 观察分类准确率

我固定了网络层数为 2 层, 分别测试了 3x3, 5x5, 7x7, 9x9 四种不同大小的卷积核。

关键代码 (7x7卷积核 task3_kernel_impact/cnn_kernel_7.py):

```
1  # CNN with Kernel Size 7
2  class ConvNetKernel7(nn.Module):
3      def __init__(self):
4          super(ConvNetKernel7, self).__init__()
5          # Layer 1: Kernel 7
6          self.layer1 = nn.Sequential(
7              nn.Conv2d(1, 16, kernel_size=7, stride=1, padding=3),
8              nn.BatchNorm2d(16),
9              nn.ReLU(),
10             nn.MaxPool2d(kernel_size=2, stride=2))
11         # Layer 2: Kernel 7
12         self.layer2 = nn.Sequential(
13             nn.Conv2d(16, 32, kernel_size=7, stride=1, padding=3),
14             nn.BatchNorm2d(32),
15             nn.ReLU(),
16             nn.MaxPool2d(kernel_size=2, stride=2))
17         # ...
```

实验结果:

```
1  Running Task 3: Kernel Impact (Kernel 3x3)...
2  Accuracy of CNN (Kernel 3x3) on 10000 test images: 98.51 %
3
4  Running Task 3: Kernel Impact (Kernel 5x5)...
5  Accuracy of CNN (Kernel 5x5) on 10000 test images: 98.72 %
6
7  Running Task 3: Kernel Impact (Kernel 7x7)...
8  Accuracy of CNN (Kernel 7x7) on 10000 test images: 99.27 %
9
10 Running Task 3: Kernel Impact (Kernel 9x9)...
11 Accuracy of CNN (Kernel 9x9) on 10000 test images: 99.13 %
```

实验分析:

- 小卷积核:** 3x3 卷积核的准确率相对较低 (98.51%)。这可能是因为层数较少 (仅2层) 的情况下, 3x3 的感受野较小, 难以一次性捕捉到数字的整体结构。
- 大卷积核优势:** 7x7 卷积核取得了最高的准确率 (99.27%)。对于 28x28 的 MNIST 图像, 较大的卷积核可以直接覆盖图像的显著部分, 有助于提取全局形状特征。
- 过大卷积核:** 9x9 卷积核的准确率略有下降 (99.13%), 说明盲目增大卷积核并不总是有效的, 可能会引入过多的参数导致过拟合, 或者忽略了局部的细节特征。

实验总结

- Loss 收敛性:** CNN 的训练过程是有效的, Loss 随迭代次数稳定下降。
- 深度至关重要:** 增加网络深度 (层数) 通常能显著提升模型的特征提取能力和分类准确率, 本实验中 5 层网络表现最佳。
- 卷积核选择:** 卷积核大小对性能有显著影响。对于简单的小图像数据集 (如 MNIST), 适当增大卷积核 (如 7x7) 可以获得比经典 3x3 或 5x5 更好的效果, 但过大的卷积核可能会带来收益递减。